AI 驱动研发体系的实践和思考

原创 营销&交易技术 大淘宝技术 2026年9月2日 01:08 浙江 136人

本文总结了团队在 AI 驱动研发体系的实践探索，从 AI 辅助编程发展到完整的 AI 驱动研发体系。文章指出业务AI研发的真正瓶颈是上下文而非模型能力，提出"本地优先：Agent + 文件夹 + Git"理念，通过 Price360-KB 项目 Harness 将业务知识、源代码、项目规则和验证证据组织在同一工作空间。实践表明迭代过程本身就是知识飞轮，项目无需完美知识库即可启动，在真实产品迭代中持续完善。文章阐述了项目 Harness 的三层结构设计、机器可读的迭代协议以及人负责决策而 Agent 负责执行完整的工作模式。随着模型和通用 Agent 能力提升，项目 Harness 将收缩，但业务知识治理和质量责任不会消失，未来技术人员将更接近 FDE 角色，需深入理解业务、设计项目规则并对端到端交付结果负责。

01 从 AI 辅助编程到 AI 驱动的研发体系

今年 4 月，受到 Karpathy LLM Wiki 思路的启发，开始尝试用AI来驱动研发体系。经过了4个月的实践，我们团队的研发体系发生了巨大的变化，从AI辅助编程转向了AI驱动的研发体系。再此总结下一路实践过来的思考，然后重新出发。

AI 辅助编程：提升局部动作效率

AI 辅助编程主要是靠人在提示词输入框中将代码+要求告诉 AI 怎么去修改，业务规则都是通过提示词输入框给到 AI，新开一个会话又要重复一次输入。

Coding Agent：从生成内容走向执行任务

当 AI 可以操作文件、代码、Git 和外部工具，它就不再只是辅助，而是能够检索、修改文件并提交。此时可以用一个公式来理解：`Agent = Model + Harness`，模型之外都是 Harness。

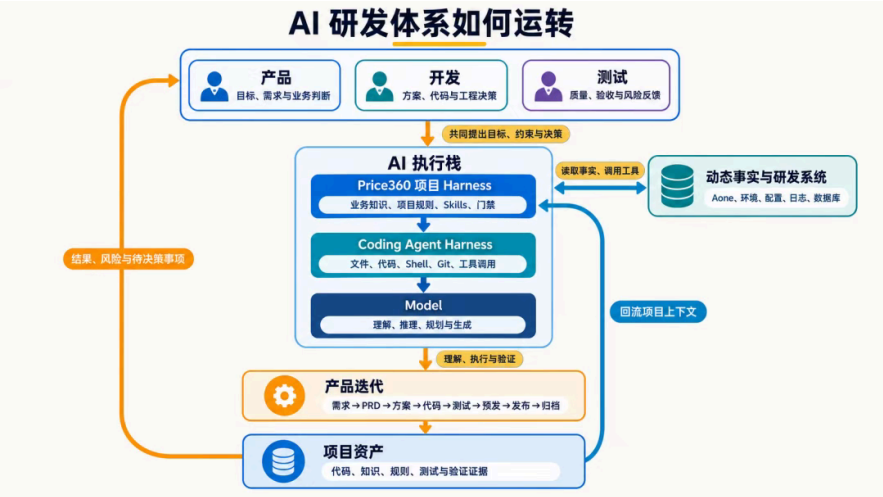
Model 提供理解、推理、规划和生成能力，Coding Agent Harness 提供AgentLoop、文件操作、工具使用和环境连接，但缺少业务知识、流程约束和完成标准，Agent 依然需要人反复补充背景，也可能把命令执行成功误判为业务交付完成。

项目 Harness：让 AI 进入完整研发闭环

AI 驱动的研发体系需要在通用 Coding Agent 之上增加项目 Harness。Price360-KB 是我们在价格力业务中的内部实践：它把业务知识、源代码、项目规则、流程 Skill、质量门禁和验证证据放在同一个工作空间，并通过 CLI、MCP 连接工作项、环境、配置、日志和数据库等动态事实。这时上面的公式变成了下面这样：

```
业务研发Agent= Model + Coding Agent通用Harness + 项目Harness
```

现在，产品、开发、测试同学都在这套体系里共同提出目标、约束与决策，人的重心也从逐步指导操作，转向业务判断、方案选择、结果验收和高风险授权。



02 从 LLM Wiki 到 AI 驱动研发体系的思考

在介绍实践之前，先梳理这一路最重要的思考。内部仓库无法直接开放给外部读者，因此第三章会抽取其中已经落地的核心机制。比起照搬目录和工具，我更希望读者理解这些机制为什么存在，以及怎样迁移到自己的项目。

真正的瓶颈是上下文

模型能力在快速提升，但它进入一个真实业务系统后，仍然会遭遇大量信息缺口。它不知道历史上为什么这样设计，不知道某个规则只适用于哪些业务，也不知道某段代码是当前链路还是

已经废弃的实现。

如果每次都靠人在对话里重新讲解，AI 的表现就会受制于当前对话长度和个人表达能力。同一个问题，换一次会话、换一个人或换一个 Agent，结果可能完全不同。

业务 AI 研发的上限不只由模型决定。模型决定通用能力，上下文决定它在具体项目中能走多远。

本地优先：Agent + 文件夹 + Git 🌟

尽可能将项目需要的所有信息放进同一个文件夹，然后使用 Git 管理，Coding Agent 可以非常天然的操作 文件系统，Git 再补上版本、分支、评审、来源和回滚，不需要建设一套复杂平台和操作界面，所有的操作都通过 Coding Agent 打开文件夹对话进行，不要尝试手动修改任何一个字。

本地优先不等于把所有信息都保存到本地。业务知识、项目规则、源代码和迭代文档适合进入 Git，Aone 状态、测试环境、日志、数据库和配置中心等动态事实仍然从权威系统实时查询。稳定上下文进入 Git，动态事实留在它们应该在的地方提供 MCP 或者 CLI 给 Agent 使用，两者在 Agent 执行时汇合。

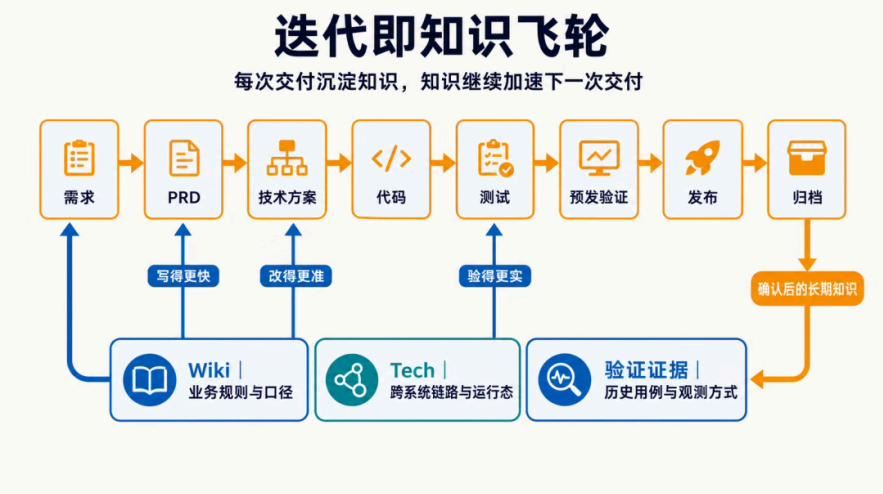
老系统的冷启动

新项目可以从第一天就按 AI 友好的方式组织，老系统不行。高价率运营涉及多个前端、后端和离线数据工程，知识分散，文档完整度也不一致。我们可以将原有的资料和代码关联到项目中，然后再设计一个清洗SKILL，从代码和凌乱的资料文档中清洗出结构化的知识。

Price360-KB 通过 Git submodule 把相关代码仓库关联到 `src/` 目录。之前人工编写的文档资料等可以关联到 `raw/` 目录。有了初步的知识沉淀后，接下去可以设计采访稿，通过钉钉AI听记对核心开发、产品人员进行采访，然后将采访原稿给到 Coding Agent，让他整理到项目知识库，补齐只存在人脑海中的业务知识。

不要等到知识库非常完善才开始使用起来，核心是在AI研发体系的每次需求迭代过程中去完善知识库，形成一个知识飞轮。

迭代过程本身就是知识飞轮🌟



这条链路里，每个阶段都在为下一个阶段生产上下文。PRD 补充业务定义和验收标准，技术方案记录系统链路，开发和测试继续校验它们，归档时再把确认后的长期知识回流到 `wiki` 和 `tech`。所有的改动都通过Git管理，一个完整的迭代保证了代码和知识库一致性。

知识又会反过来影响下一轮迭代。有了业务规则，Agent 写 PRD 时就不需要从零推测概念和口径，有了跨系统技术链路，技术方案就更容易找到真正的改造点，有了历史用例和观测方式，测试也不用每次重新发明验证方法。

因此，项目开始时不需要一个完美的知识库。知识建设可以成为真实产品迭代的副产物，再逐步变成后续迭代的加速器。

为什么先做文件 Wiki

LLM Wiki、RAG 和本体论并不是三个互斥方案。它们解决的问题不同：文件 Wiki 处理知识如何维护、评审和版本化，RAG 解决如何找到相关内容，本体负责表达实体、关系和领域语义。

我们优先选择文件 Wiki，是因为知识更新路径足够短。代码、文档、测试和索引可以进入同一个 MR，一起评审，并在合并后共享同一个版本。一次迭代可以同时更新功能和知识，减少代码已经改变、文档仍停留在旧版本的情况。

如果把外部 RAG 平台或本体图谱直接当作主知识源，代码变化之后还要经过同步、切片、萃取、索引或图谱发布。在当前的研发流程里，这会拉长更新链路，增加知识与真实代码发生漂移的机会。

本体论在知识应用上面存在优势，但是缺少知识的生产和维护，而 price360-kb 这种模式很好的解决了知识生产和维护，让产品、运营、开发、测试等所有角色都共同来维护一个知识库。



知识库应该保存什么

知识库很容易走向另一个极端：把每个类和每个方法重新解释一遍。这类文档生成很快，失效也很快，最后变成一份与代码竞争的实现说明。

Price360-KB 将长期知识分为 `wiki` 和 `tech`。

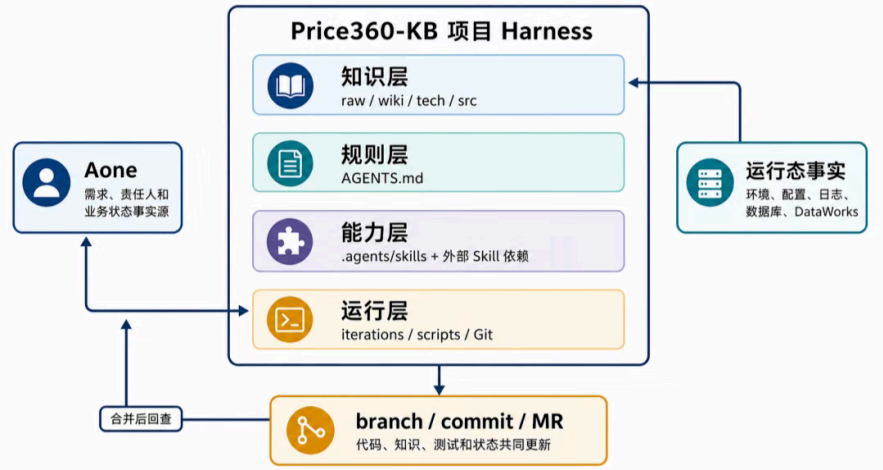
`wiki` 面向产品、运营、测试和 Agent，保存业务概念、规则、口径、角色、流程和异常处理。业务知识通常分散且缺少持续维护，所以它既可以来自已有产品材料，也可以从代码中挖掘。产品在 PRD 阶段补充历史逻辑，开发和测试阶段继续校验，迭代完成后再归档到 Wiki。

`tech` 保存代码无法独立表达、但会影响 Agent 判断的技术上下文，例如跨系统上下游链路、接口和数据之间的关系、新旧链路切换、废弃状态、运行态拓扑和观测方式。具体方法逻辑以代码为准，技术文档只需说明调用哪个接口、哪个方法，以及它在整条链路中的位置。只有特别重要、反复使用的逻辑，才值得保留稳定摘要，避免 Agent 每次重新读取大量代码。

知识库不应该复制代码。它保存的，是代码之外会影响业务和技术判断的信息。

03 Price360-KB 项目 Harness

Price360-KB 不是一套独立研发平台，而是一个可以被 Coding Agent 直接打开和执行的 Git 工作空间。它把稳定上下文、迭代产物、项目规则和自动化能力组织在一起，再连接外部系统中的动态事实。项目结构可以简化为下图。



核心目录结构如下。真实项目中还有模板、客户端配置和辅助工具，这里只保留理解 Harness 所需的部分。

```
price360-kb/
├── AGENTS.md                    # 项目协作协议与总入口
├── iterations/                  # 每个需求的 PRD、方案、测试与归档
├── wiki/                        # 面向产品、运营和测试的业务知识
├── tech/                        # 跨系统链路与关键技术上下文
├── raw/                         # 未经改写的原始事实材料
├── src/                         # 关联的业务代码仓库
├── olap/                        # 指标、表结构与分析 SQL
├── briefs/                      # 基于事实源生成的专题说明
├── outputs/                     # 可重建的索引和分析结果
└── .agents/
    ├── skills/                  # PRD、方案、测试、排障、归档等领域能力
    ├── scripts/                 # 工作区、校验、同步和发布门禁
    ├── repositories.json        # 项目关联的代码仓库声明
    └── skill-dependencies.json  # 公共 Skill 依赖声明
```

这棵目录树同时表达了事实边界和执行边界：知识、代码、迭代证据各有固定位置，`AGENTS.md` 负责统一规则，`.agents` 负责把规则变成可执行能力。Agent 不需要先学习一套独立平台界面，打开项目后就能沿着相同结构检索上下文、推进任务并留下证据。

文件不是资料堆积，而是有来源的上下文网络

工作空间里的内容按用途分层：`raw` 保存未经改写的原始材料，`wiki` 保存业务概念、规则和口径，`tech` 保存跨系统链路与关键技术上下文，`src` 关联实际代码，`iterations` 保

存每次需求从 PRD 到归档的过程资产。不同内容有不同的事实边界，Agent 不能把一份会议记录直接当成已经确认的业务规则，也不能用旧文档覆盖当前代码事实。

一份知识文件由正文、Metadata、链接、事实来源和 Git 版本共同构成。

例如，一份业务规则文档可以使用下面的 Metadata：

```
---
title: "价格归因领域知识"
category: "规则"
tags: ["价格", "归因", "xx原因"]
status: "有效"
version: "v1.1"
source:
  - "raw/existing_docs/xxx说明.md"
  - "tech/01-链路详解/xxx处理管线.md"
---
```

`title`、`category`、`tags` 和 `status` 让 Agent 在读取全文之前就能判断文档是什么、是否适用。`source` 是事实来源字段，用来说明当前知识来自哪份原始材料、技术分析或代码版本。

如果正文中的关联其他知识文件，则采用相对文件路径进行关联，使关联概念可以快速检索到。`tech` 文档再通过 `wiki_ref` 关联对应的业务知识，形成从业务规则到技术链路、再到源代码的路径。Agent 既可以从业务问题定位技术入口，也可以从代码变化反查可能受影响的业务规则。

知识生成统一经过项目 Skill，补齐标签、来源和索引；定期健康检查再处理迭代积累下来的重复、冲突、失效链接和无来源结论。

协议、能力和执行门禁分开维护

项目 Harness 不是一份越来越长的提示词。Price360-KB 把它分成三层：

- `AGENTS.md` 是项目级协作协议，定义事实源、研发阶段、人工决策点、授权边界和完成条件。
- `.agents/skills` 是可组合的领域能力，分别处理 PRD、技术方案、测试设计、回放、排障和知识归档等任务。
- `.agents/scripts` 负责适合确定性执行的动作，例如检查工作区、校验产物格式、同步状态、提交推送和发布后回查。

这样分层以后，规则负责说明“应该怎样做”，Skill 负责组织一类任务，脚本负责把不能靠语言自觉保证的门禁执行到底。模型或 Coding Agent 产品可以替换，项目自己的协作协议仍然保留。

公共能力也不复制进每个项目。代码平台、中间件和数据开发等通用 Skill 通过依赖清单声明来源和兼容版本，项目 Skill 只保留业务编排。公共能力可以统一升级，业务团队也能继续维护自己的领域规则。

把一次迭代写成机器可读协议

每个需求都有独立迭代目录，保存 `prd.md`、`solution.md`、`test/design.md`、`test/cases*.md`、`test/report.md` 和 `archive.md`。这些文件不是交付结束后的总结，而是阶段之间的接口。

PRD 用 REQ 和 AC 描述需求与验收标准；技术方案把它们映射到实现任务和可观测点；测试设计与用例再用 TC 建立覆盖关系。状态、版本、依赖和证据字段采用固定格式，Agent 可以检查“某个验收标准是否有方案承接、是否有测试覆盖、用例是否仍对应当前方案版本”，而不是靠人通读多份文档后凭印象判断。

流程按“需求 → PRD → 技术方案 → 测试设计 → 代码 → 预发验证 → 发布归档”推进。阶段不是自动越过的：PRD 未确认，不写技术方案；方案和测试设计未确认，不进入代码实现；没有真实测试证据，不进入发布准备。上游内容发生实质变化时，下游产物必须更新，旧证据也不能继续沿用。

Git 保存资产，外部系统提供动态事实

稳定上下文进入 Git，动态事实仍从权威系统实时读取。工作项平台保存需求状态和负责人，代码平台保存分支、提交与评审，测试环境、日志、数据库和配置中心提供运行时现场。迭代 ID、工作项 ID、分支、commit 和 MR 把这些事实串起来。

这一区分很重要。配置文件里写着某个环境，不代表当前代码已经部署到那里；接口返回成功，也不代表异步消费、内部判断和落库副作用已经发生。Price360-KB 要求测试先绑定当前分支对应的真实环境，再按场景验证返回值、日志、调用链或数据库结果，并把 traceId、执行时间和断言结果回填到测试资产中。无法取得稳定证据的场景就保留为人工验证，不能包装成自动化通过。

发布也遵循同样的完成标准。代码合入主干只是其中一步，之后还要回查工作项和协同状态。只有代码版本、业务状态和交付证据一致，一次迭代才算真正结束。

人负责决策，Agent 负责把决策执行完整

在这套流程中，Agent 负责检索上下文、生成产物、执行检查、调用工具和收集证据；人负责描述需求、选择方案、确认测试设计、验收预发结果，以及授权合并和生产发布。高风险动作不会因为前面已经得到过一次概括性同意就自动执行。

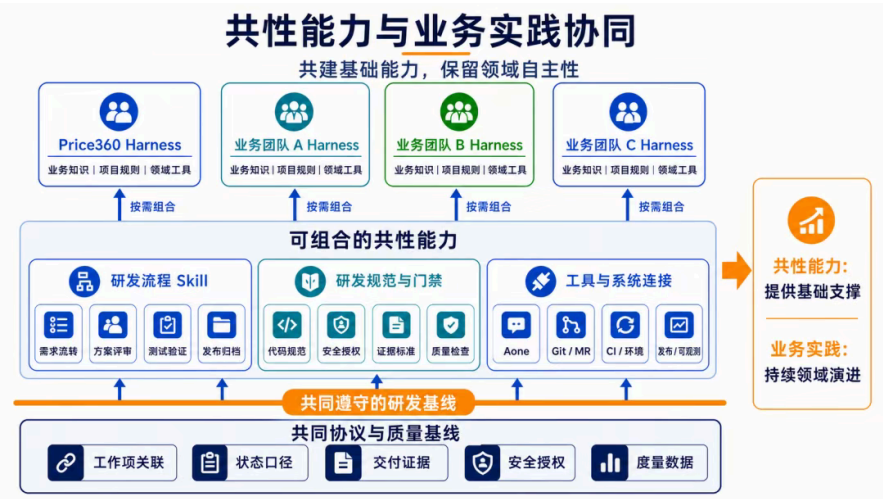
这种分工并没有把人排除在流程之外，而是减少人对操作步骤的持续看护。人给出业务判断后，Harness 负责把它传递到后续文件、代码、测试和状态中，并在缺少证据或越过授权边界时阻断流程。

Price360-KB 的 Harness 也不是一开始就设计好的。多需求并行污染目录后，我们增加了工作区隔离；接口成功却无法证明异步链路完成后，我们增加了可观测性契约；外部状态提前更新造成不一致后，我们把发布改成合并后回查。它的演进方式很朴素：把每次真实交付暴露的问题，变成下一次可以复用的规则、检查或工具。项目在交付业务功能的同时，也在改进自己下一次交付的方式。

04 共性能力与业务实践如何协同

从效率、成本、质量、团队管理方面来思考，未来 AI 研发体系一定会出现横向公共能力建设，现在都还在探索阶段，每个团队都有自己的最佳实践，即使是在我的团队基于我的思想实践，也出现了多个独立的项目 Harness。

业务系统之间存在较大的差异性，不同系统拥有自己的知识结构、测试方法、运行环境和发布风险，因此很难用一套实现覆盖所有项目。更合理的边界，是复用已经验证的共性能力，同时允许业务团队保留贴合领域的项目 Harness。



AI 正在降低生成、修改和维护项目 Harness 的成本。业务团队可以围绕自己的知识、规则和工具持续优化，不必为了形式统一而牺牲领域适配性。但是，共同的数据协议、质量基线对大团队管理、提效是有增益的。

一种更稳妥的协作方式，是把成熟实践沉淀为可直接使用的参考实现和可组合能力。这里包括定义需求流转、方案评审、测试验证和发布归档的流程 Skill，以及代码规范、安全授权、证据标准和质量门禁。没有成熟实践的团队可以直接采用参考方案，已经有积累的团队可以按需复用，再在项目层继续迭代。

真正需要形成共识的，是工作项、代码、测试和发布之间的数据协议，以及状态、证据、安全和度量口径。只有这些数据基础一致，后续才能衡量效率、项目 Harness 质量、人的能力。

关于衡量指标，现在的 AI 代码采纳率会受很多因素影响，无法体现出人的工作质量、对 AI 的驾驭程度，更像是不得已的妥协方式。如果能建立统一 AI 研发体系数据协议，用 Aone 工作项串联从需求到上线的所有 AI Trace，Aone 节点流可以跟 Trace 中的 step 映射，那么将可以用 AI 来分析一个需求上线的质量，比如：跨对话中反复补充上下文肯定是效率不高的做法、反复纠正之前错误的对话说明项目 Harness 有待完善。同时我们也可以根据分析发现我们离自动化 AI 研发还差在哪里，哪些信息需要补齐、哪些地方必须要人来决策。

05 结语：当 Harness 不断收缩

回到开头的公式：`Agent = Model + Harness`

Model 和通用 Coding Agent 的 Harness 能力都在扩大。今天还需要项目自己维护的 Prompt、规则和工具，一部分会被模型或 Coding Agent 产品吸收。文件搜索、代码理解、Git、浏览器、工具调用和多 Agent 协作会逐步变成通用能力，业务项目需要维护的 Harness 会随之收缩。

但有几类东西不会自动消失：业务知识及其事实治理，项目规则与决策边界，领域工具和系统适配，以及项目自己的验证标准和质量责任。

这也会重新定义业务技术人员的价值。产品和技术之间的边界会继续变模糊，技术人员需要理解 AI，也要理解业务，能够设计项目规则和领域工具，并对端到端的交付结果负责。

我们的角色将更接近 FDE（Forward Deployed Engineer）：深入业务现场，把问题、知识、系统和交付结果连接起来。当模型和通用 Agent 能力继续扩大，业务技术人员真正需要守住的是对业务的理解、对系统的连接能力，以及对结果的责任。

06 团队介绍

本文作者默达，来自淘天集团–营销&交易技术团队。团队围绕淘系价格力建设，持续完善价格基础能力、经营工具和技术平台，帮助业务理解价格表现、发现经营问题并推动价格优化。团队也在探索 AI 在业务知识沉淀、研发协作和端到端交付中的应用，Price360-KB 是其中一项实践。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

966篇原创内容

公众号

阅读 1.5万

留言

写留言

还没有留言，来写首评